

TEAMFLOW – DATABASE SCHEMA

Project Title

TeamFlow – MERN Stack Project Management Application

1. Introduction

The TeamFlow application uses **MongoDB Atlas** as its primary database. MongoDB is a NoSQL document-oriented database that stores data in JSON-like documents (BSON). The application uses **Mongoose** as the Object Data Modeling (ODM) library to define schemas and manage database operations.

The current implementation consists of three primary collections:

- Users
- Projects
- Tasks

Relationships between collections are maintained using **MongoDB ObjectId references**, allowing efficient retrieval of related data while maintaining data consistency.

2. Users Collection

The **Users** collection stores information about registered users who can log in and access the application.

Fields

Field	Data Type	Description
_id	ObjectId	Unique identifier
name	String	User's full name
email	String	Unique email address
password	String	Encrypted password
role	String	User role (Admin / Employee)
createdAt	Date	Creation timestamp
updatedAt	Date	Last updated timestamp

Schema Representation

```
{
  _id:ObjectId,
  name:String,
  email:String,
```

```
password:String,  
role:String,  
createdAt:Date,  
updatedAt:Date  
}
```

3. Projects Collection

The **Projects** collection stores information related to projects created by authenticated users.

Each project belongs to one creator and may contain multiple members.

Fields

Field	Data Type	Description
_id	ObjectId	Unique identifier
title	String	Project title
description	String	Project description
createdBy	ObjectId	Reference to User
members	Array<ObjectId>	Project members
createdAt	Date	Creation timestamp
updatedAt	Date	Last updated timestamp

Schema Representation

```
{  
  _id:ObjectId,  
  title:String,  
  description:String,  
  createdBy:ObjectId,  
  members:[ObjectId],  
  createdAt:Date,  
  updatedAt:Date  
}
```

4. Tasks Collection

The **Tasks** collection stores individual tasks associated with projects.

Each task belongs to exactly one project.

Fields

Field	Data Type	Description
_id	ObjectId	Unique identifier
title	String	Task title
description	String	Task description
project	ObjectId	Reference to Project
status	String	Pending / In Progress / Completed
dueDate	Date	Task deadline
createdAt	Date	Creation timestamp
updatedAt	Date	Last updated timestamp

Schema Representation

{	
_id:	ObjectId,
title:	String,
description:	String,
project:	ObjectId,
status:	String,
dueDate:	Date,
createdAt:	Date,
updatedAt:	Date
}	

5. Entity Relationships

The TeamFlow database follows a simple relational structure using MongoDB references.



Relationship Description

User → Project

- One user can create multiple projects.

- Every project has exactly one creator.
 - Relationship: **One-to-Many (1:N)**
-

Project → Task

- One project can contain multiple tasks.
 - Every task belongs to exactly one project.
 - Relationship: **One-to-Many (1:N)**
-

Project → Members

- One project can have multiple members.
 - A user may belong to multiple projects.
 - Implemented using an array of ObjectId references.
-

6. Database Collections

Database	Name
teamflow	
Collections	
users	
projects	
tasks	

7. Database Design Decisions

The following design decisions were made while designing the database.

MongoDB Atlas

MongoDB Atlas was selected because it provides:

- Cloud-hosted database
- Automatic backups
- Scalability
- High availability

- Easy deployment
-

Mongoose ODM

Mongoose was used because it provides:

- Schema validation
 - Object modeling
 - Middleware support
 - Relationship handling
 - Cleaner code
-

ObjectId References

Instead of embedding all information inside documents, ObjectId references were used between collections.

Advantages include:

- Reduced data duplication
 - Easier maintenance
 - Better scalability
 - Faster updates
-

Automatic Timestamps

Each collection uses:

`createdAt`

`updatedAt`

These timestamps help maintain audit information and simplify record tracking.

8. Conclusion

The TeamFlow database follows a simple and scalable NoSQL design consisting of three collections: **Users**, **Projects**, and **Tasks**. Relationships are maintained using MongoDB ObjectId references, ensuring data consistency while supporting efficient CRUD operations. The combination of MongoDB Atlas and Mongoose provides flexibility, scalability, and ease of maintenance, making the database suitable for future enhancements such as notifications, file attachments, activity logs, and role-based access control.